

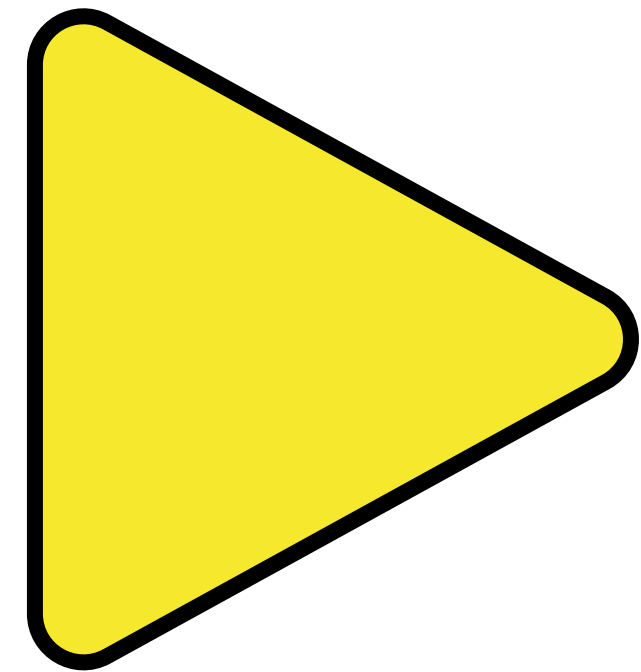
Example-guided AI- Collaborative Test-driven Coding (EXACT)

// Ferdinand Ade & Marco Emrich



<https://leanpub.com/exact-coding>

— E-Book



Vibe Coding

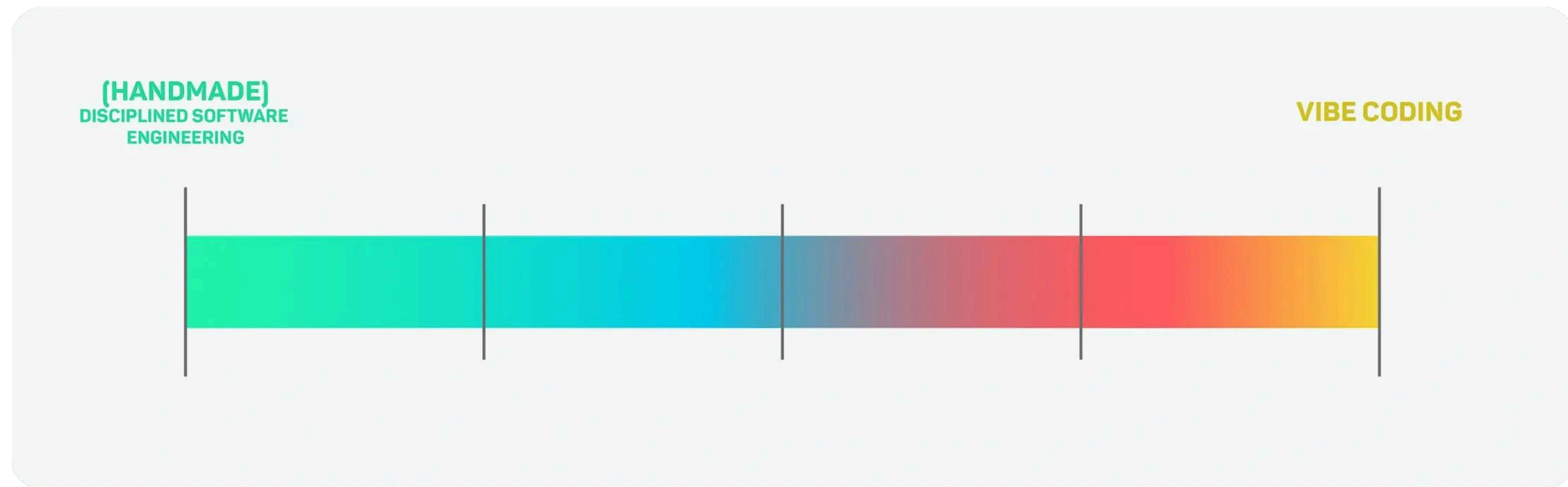
01

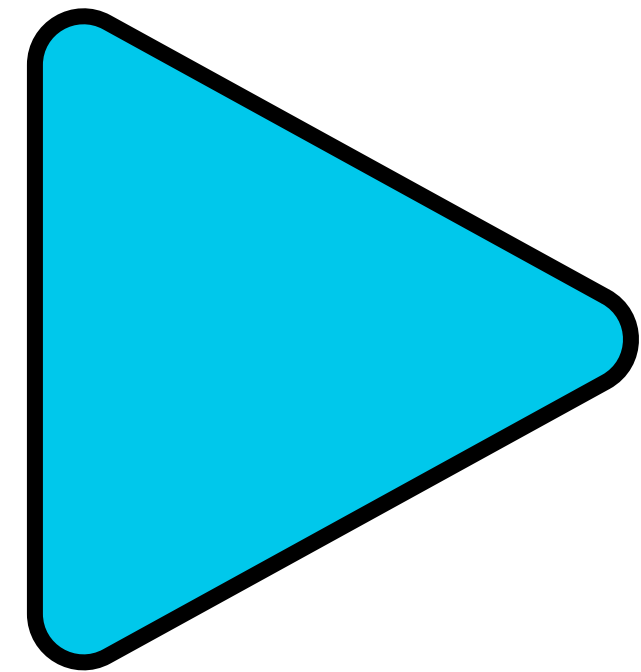
// Vibe Coding - Gone wrong



01

// Vibe Coding vs Handmade





Example Mapping

// Example Mapping WHAT's that ?!

- **A Collaborative Discovery Technique:** A structured conversation used by Agile teams to clarify requirements and reach a shared understanding before development starts.
- **The "Four-Color" Framework:** A visual method using colored cards—Yellow (Stories), Blue (Rules), Green (Examples), and Red (Questions)—to organize logic.
- **A Refinement Tool:** A quick (typically 20-30 minute) session that breaks down a User Story into its specific business rules and concrete edge cases.
- **A Bridge to Testing:** A way to turn abstract ideas into specific "Green" examples that serve as the foundation for automated acceptance tests.

// Example Mapping WHY ?!

- **Builds Shared Understanding:** Aligns the "Three Amigos" (Business, Dev, QA) on exactly how a feature should behave, preventing expensive mid-sprint misunderstandings.
- **Exposes Missing Information:** Quickly identifies gaps in logic or unanswered questions (Red cards) before any code is written, reducing technical debt and rework.
- **Simplifies Test Creation:** Provides a ready-made list of concrete scenarios (Green cards) that can be directly converted into automated acceptance tests or Gherkin scripts.
- **Sizes Stories Effectively:** Visually reveals if a story is too complex; if there are too many Blue cards (rules), it's a clear sign the story needs to be split into smaller tasks.

02

// Example Mapping HOW

Example Mapping

Developed together as a team

Story: Customer books a meeting

Rule 1
only Mo-Fr 8-18

Rule 2
min. 24h Lead time

Example
Mi 14:00 ✓

Example
Sa 10:00 X

Example
in 2h X

? Closed on the holiday too ?

What it becomes

Story context + executable tests

Story Description:

Feature: Book Meeting

As a customer, I want to schedule an appointment.

Rules:

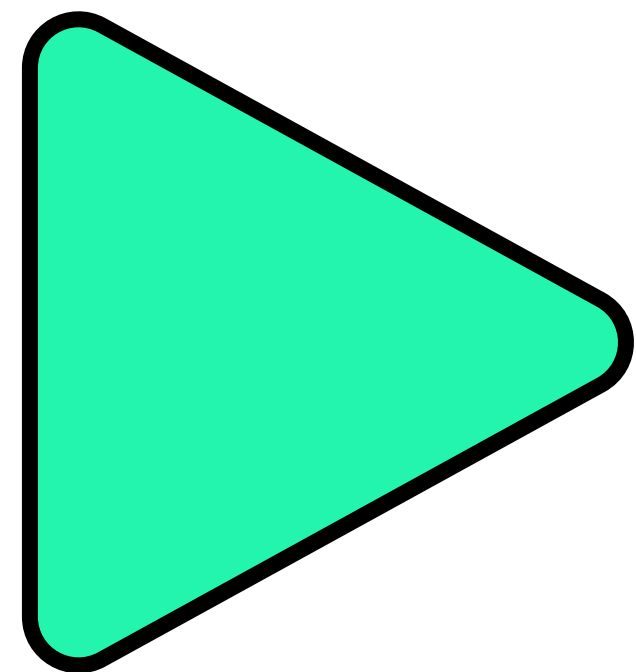
Rule 1
only Mo-Fr 8-18

Rule 2
min. 24h Lead...

Edge Case Tests (Gherkin)

Scenario: Sa rejected [Rule 1]
When I select Sa 10:00
Then it is rejected
And message: "Weekdays only"

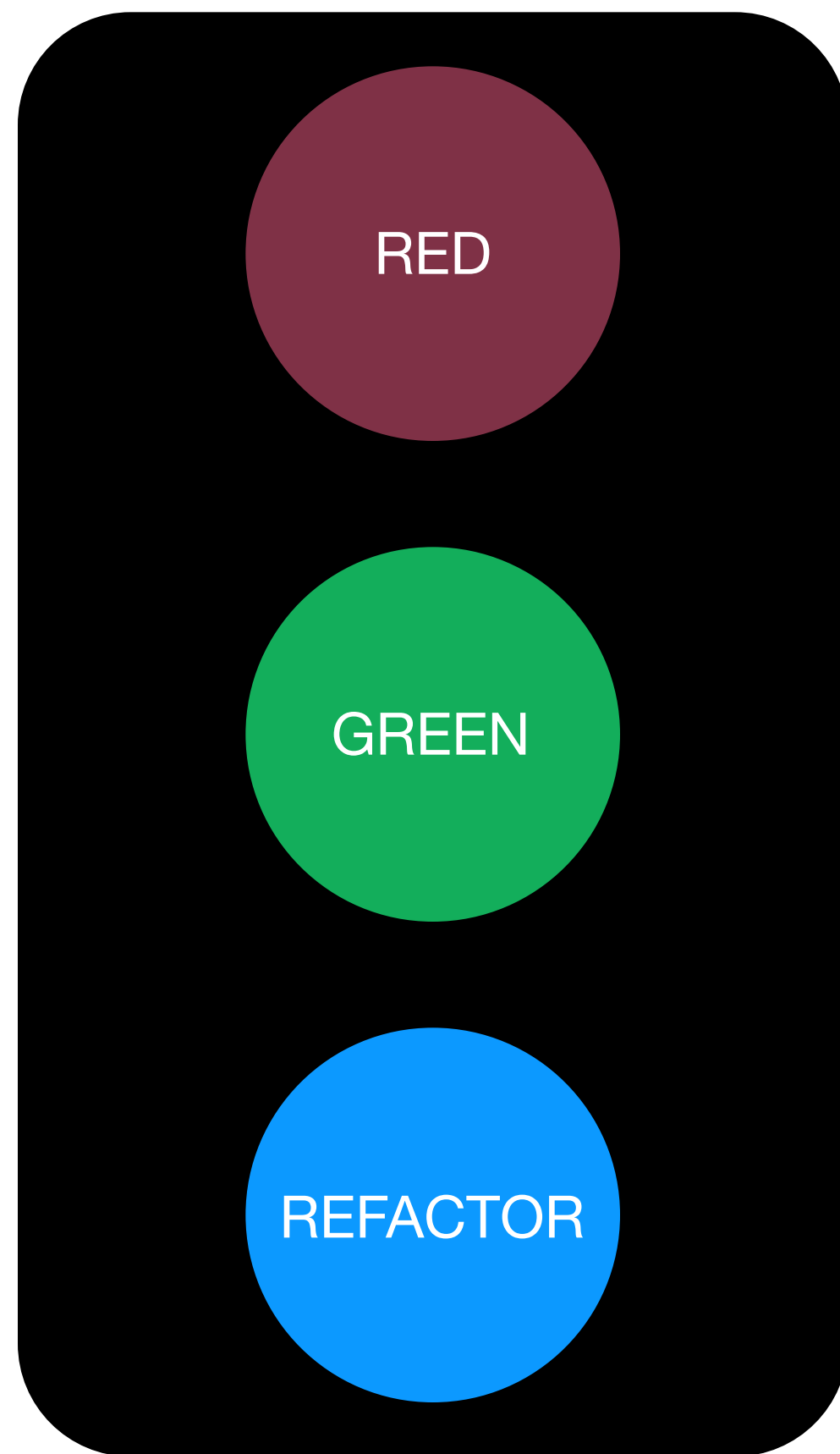
Scenario: rejected in 2h [Rule 2]
When start in 2 hours
Then it is rejected
And message: "Lead time 24h"



**AI
TDD**

03

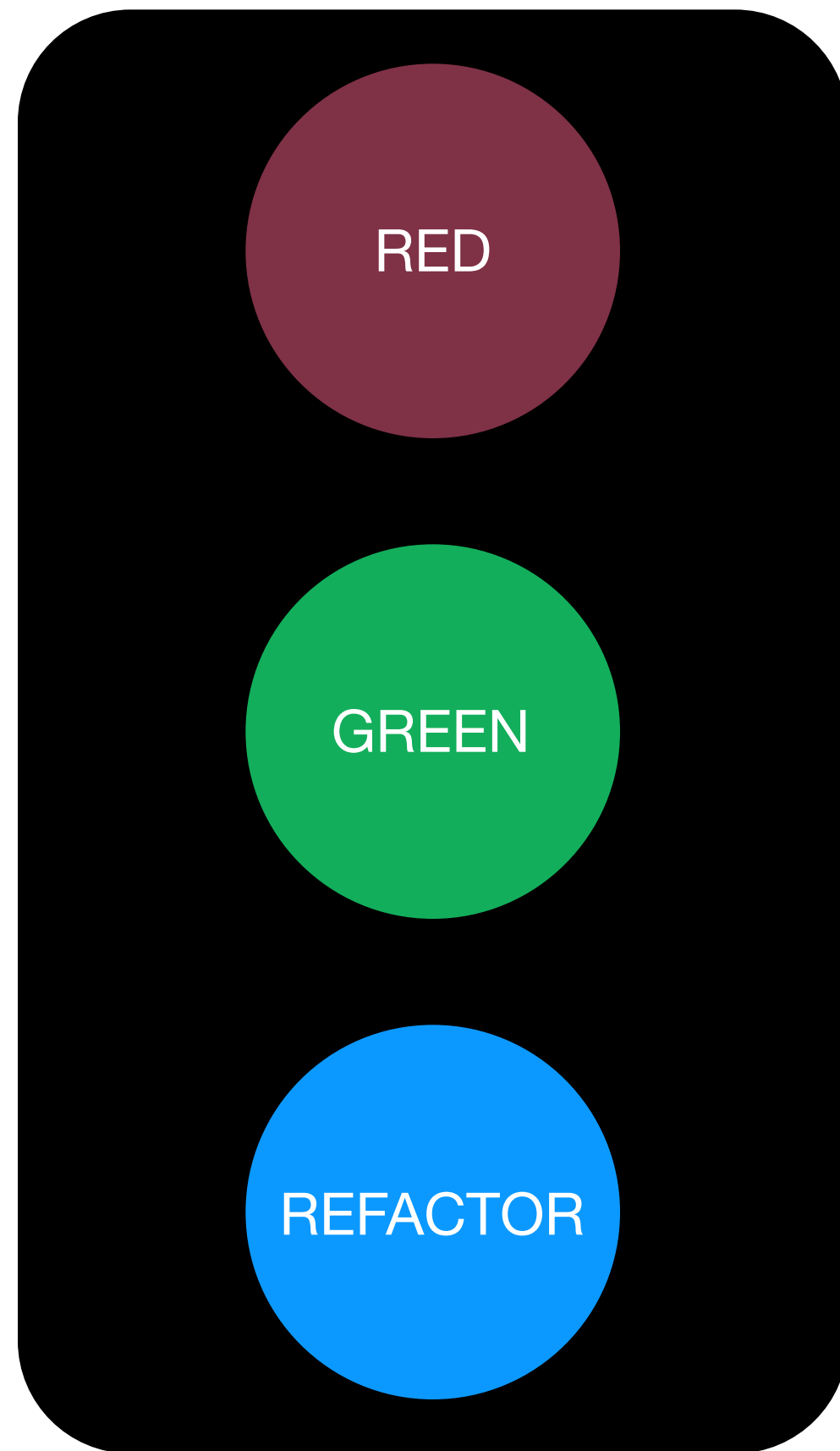
// TDD WHAT ?!



- **1. Write Test:** Make sure it is red(breaks)
- **2. Make Green:** Implement the code so test is green
- **3. Refactor:** Simplify the implementation, refactor.
- **Next Test**

03

// AI TDD WHAT ?!



- **1. Write Test:** Make sure it is red(breaks) - **AI**
- **2. Make Green:** Implement the code so test is green - **AI**
- **3. Refactor:** Simplify the implementation, refactor. - **AI**
- **Next Test**

// AI TDD WHY ?!

- **Simplifies Code Quality:** TDD workflows consistently produce simpler, less complex code than non-TDD approaches.
- **Prevents Over-Engineering:** Using a checklist without TDD actually leads to the most complex code; TDD forces a "minimalist" focus.
- **Enforces AI Discipline:** Structured TDD prevents AI from "vibe coding" or over-implementing, ensuring it only writes the code necessary to pass a test.
- **Verifies Logic:** Requiring an AI to predict test failures ensures its "mental model" matches the actual code state, preventing it from losing the thread.

// AI TDD WHY - The experiment

- **A controlled lab:** The same coding katas, built over and over, varying one thing at a time
- Runs 659 total, Claude Code CLI, headless, no human in the loop, Anthropic only (Opus 4.6/4.7, Sonnet 4.6, Haiku 4.5)
- **Enforces AI Discipline:** Structured TDD prevents AI from "vibe coding" or over-implementing, ensuring it only writes the code necessary to pass a test.
- **Verifies Logic:** Requiring an AI to predict test failures ensures its "mental model" matches the actual code state, preventing it from losing the thread.

03

// AI TDD WHY - The three axes verified independently

Workflow

6 GENERATIONS · V1 → V8

- **Vibe-coding** baselines (v1/v2) — just “implement X”, no tests
- **Strict TDD** with Red / Green / Refactor (v4–v6)
- **Hybrid** & isolated refactor subagents (v6/v7)
- **Delayed refactor** after vibe-coding (v8)

Prompt style

3 VARIANTS

- **Prose** — rules described in words, no examples
- **User-story** — “As a... I want...”, no examples
- **Example-Mapping** — rules + concrete I/O examples

Kata × Model

5 KATAS · THINKING ON/OFF

- **Known** tasks (game-of-life) the model saw in training
- **Novel** domain (claim-office) — an unseen insurance spec
- Each model run with **thinking on and off**

PER RUN

Agent builds the kata
headless, fresh container



Tests + analyzers run
Vitest, ESLint, SonarJS, Stryker



Metrics extracted
quality, correctness, cost



Aggregated per kata
workflow × model tables

// AI TDD WHY - What they measured

Correctness

tests_passing

Inner check: do the agent's own Vitest tests go green? A failing run never counts as a solution.

verification_pct KEY

Outer check: share of an **external** acceptance suite the agent never saw that passes (0–1). The honest correctness signal.

Efficiency / cost

total_tokens

All tokens across the run incl. subagents — the price of the result.

duration_seconds

Wall-clock time end to end.

Code quality & complexity

cognitive_max MAIN

SonarSource Cognitive Complexity of the worst function — weights nesting like a human reader. **The study's headline metric.**

mccabe_max cc_longest_function

Classic cyclomatic complexity & longest function — the worst spot in the code.

smell_total

All ESLint + SonarJS rule violations: magic numbers, duplication, collapsible logic...

code_mass (APP) cc_loc

Weighted size of production constructs & lines — how much code the feature really costs.

Test strength & discipline

mutation_score HIDDEN

Stryker mutates the code — do the agent's tests catch it? Never named in any prompt, so it can't be gamed (Goodhart-resistant).

Coverage

coverage_statements_pct / **branches_pct** — how much the tests exercise.

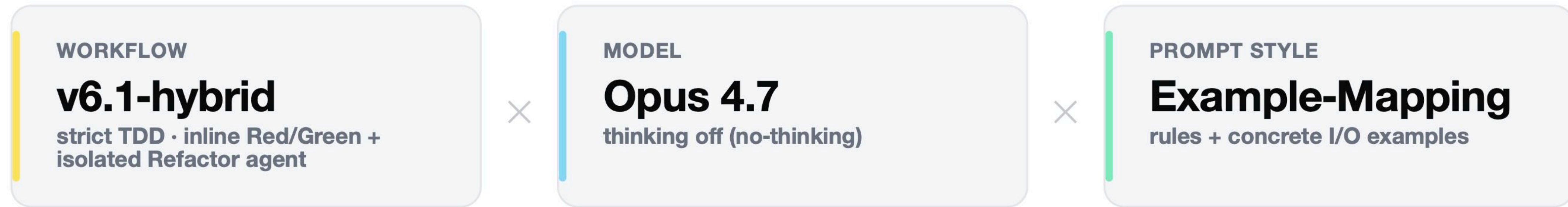
TDD discipline

cycle_count , **refactorings_applied** , red-phase failure predictions — was the process actually followed?

Direction: complexity, smells & cost — **lower is better**. Correctness, mutation score & coverage — **higher is better**.

03

// AI TDD WHY - The result



- **The workflow makes it maintainable.** v6.1-hybrid is the most **robust** structure — rank 1 or in the tight top field across every quality axis, the **only** workflow with a 0% outlier rate and no collapse on the novel kata.
- **The model sets the ceiling.** Opus 4.7 (thinking off) leads the model ranking on code quality — lowest complexity spikes and fewest smells, ahead of Sonnet 4.6 and Opus 4.6. **Thinking helped neither.**
- **Example-Mapping makes it correct.** On a novel domain it is the dominant **correctness** lever: concrete I/O examples resolve spec ambiguity, lifting external verification by **+48 to +64 points** over prose.

03

// AI TDD WHY - The result

Maintainability & complexity — lower is better

cognitive_max 0% outliers

The study's headline metric (SonarSource Cognitive Complexity). v6.1: all 10 runs stay in [1, 7].

mccabe_max **cc_longest_function**

Lowest complexity spikes — strict phase structure + refactor flattens the worst function.

smell_total ≈ 2.2

Fewest ESLint + SonarJS violations across all rules.

code_mass (APP) **cc_loc**

Compact production code — less to read, review and own.

Correctness & robustness — higher is better

verification_pct 0.28 → 1.00

External acceptance suite on the novel kata: pure vibe-coding (v1/v2) scores 0.28; strict TDD with refactor reaches a perfect 1.00.

mutation_score most stable

Goodhart-resistant hidden metric (never named in any prompt). v6.1 is the most stable workflow — lowest variance across replicates.

Prompt lever **+48–64 pp**

Example-Mapping vs prose on the novel domain (Opus 4.6 & Sonnet 4.6, no-thinking) — concrete examples kill spec ambiguity.

Quality has a price: v6.1-hybrid burns the most tokens ~33M

// Working Slot

